

UnifiedXRMotion: A Schema-Based Motion-Data Contract for Heterogeneous XR Avatar Pipelines

Anonymous Authors

Paper under double-blind review

Abstract. XR avatar-motion pipelines must combine tracking streams whose joint layouts, coordinate conventions, body coverage, validity semantics, and deployment locations differ across SDKs and devices. This heterogeneity makes it difficult to reuse the same filters, estimators, retargeters, and network wrappers when replacing a headset, adding a hand tracker, integrating a vision-based source, or moving computation between client and server. We present UnifiedXRMotion, a Unity-based prototype organized around a schema-based, validity-aware motion-data contract for heterogeneous XR avatar pipelines. The contract represents skeletal motion as addressable motion slots defined by an application schema rather than by a fixed avatar rig or a vendor SDK. Each slot stores optional pose components guarded by component-level validity flags, allowing partial trackers to publish only what they observe and allowing downstream consumers to operate over shared schema views instead of SDK-specific outputs. The prototype instantiates the contract with a 77-slot full-body schema and implements adapters for selected XR and vision-based sources, representative motion-processing consumers, and a structural local/remote node-placement wrapper. We evaluate the design through implementation coverage, adapter-mapping evidence, a within-subjects authoring case study, and application-level placement traces. The contribution is not a new tracker, IK solver, retargeting algorithm, or input-to-photon measurement; it is a concrete runtime contract and abstraction boundary for composing partial skeletal motion across heterogeneous XR avatar-motion pipelines.

Keywords: Extended reality · Avatar motion · Tracking middleware · Motion-data contract · Runtime systems · Local/remote execution

1 Introduction

XR avatars depend on motion pipelines that span tracking acquisition, skeletal representation, motion processing, retargeting, and rendering. In practice these stages are frequently tied to one SDK or one avatar asset. A hand tracker may expose a vendor-specific joint order; a body tracker may report only a subset of joints; a vision model may provide positions but no reliable rotations; a retargeter may assume a particular rig hierarchy; and an offloaded node may require a serialized representation that differs from the in-process one. The result is not merely inconvenient engineering. It weakens reproducibility and makes it hard

to compare pipelines, replace trackers, or move computation between client and server without rewriting glue code.

Existing XR APIs and tracking middleware address important parts of this problem. OpenXR standardizes access to XR devices and extensions such as hand tracking [?,?]. Earlier VR middleware, including VRPN [?], OpenTracker [?], and Ubiquitous Tracking [?], showed that device abstraction, dataflow, and distributed tracking are valuable architectural tools. UnifiedXRMotion builds on this lesson at a more specific layer: avatar-motion pipelines that must carry partial skeletal observations through filters, estimators, retargeters, record/replay utilities, and optional network wrappers.

The central design choice is to separate *slot identity* from *tracking availability*. A skeleton schema defines addressable motion slots and parent relations; a tracker publishes the subset it observes; each pose component is guarded by a validity flag; and downstream nodes consume only the components they require. This makes source-specific coordinate and validity semantics an adapter concern, while consumers operate over schema views rather than vendor SDK outputs. In our Unity prototype, this contract is instantiated with a 77-slot full-body schema covering torso, limbs, head, and articulated hands. The slot count is one prototype schema instantiation, not a universal skeleton standard. The same contract can be instantiated with smaller or larger schemas when adapters and consumers agree on the schema.

We make three contributions:

- **A schema-based, validity-aware motion-data contract** for XR skeletal motion, including addressable motion slots, parent relations, region views, optional pose components, binary validity flags, in-process buffer views, and validity-aware serialization.
- **An adapter boundary for heterogeneous tracking sources** that maps SDK-specific outputs into the shared contract. The prototype includes adapters for selected XR and vision-based sources, but we distinguish the reusable boundary from vendor-specific wrapper code.
- **A local/remote node-placement abstraction** that allows a motion-processing node to be wrapped for client-local, client-to-server, server-local, or server-to-client placement. We use application-level latency traces as a placement case study, not as a vendor-baseline performance claim.

2 Related Work

Tracking middleware and runtime abstractions. Device-independent VR tracking is not new. VRPN provides a device-independent and network-transparent interface for VR peripherals [?]. OpenTracker introduced a modular software architecture for VR interaction and tracking configurations [?]. Ubiquitous Tracking modeled distributed spatial relationships for augmented reality as a graph of tracking transformations [?]. UnifiedXRMotion inherits the lesson that modularity and abstraction matter, but it targets a more specific layer: runtime skeletal motion contracts for avatar pipelines, where missing joints, partial component

validity, region-specific views, retargeting consumers, and optional node placement must be handled explicitly.

XR APIs, avatar motion, and offloading. OpenXR standardizes device access and extensions such as hand tracking [?,?], while platform SDKs expose additional body, hand, controller, camera, or device-specific streams. These APIs tell an application how to access a source, but they do not prescribe how to represent partial skeletal motion across heterogeneous sources, mark unavailable components, or expose the same representation to filters, learned regressors, IK nodes, retargeters, and network serializers. Sparse avatar-motion reconstruction has advanced rapidly [?,?,?,?]. Avatar representation and retargeting affect embodiment and agency [?,?,?], with AvatarGo focusing on plug-and-play self-avatar setup [?]. UnifiedXRMotion does not compete with these algorithms. It provides infrastructure around them: a contract through which estimators, IK solvers, and retargeters can be represented as consumers and producers in the same runtime. For offloading, recent work such as XRgo investigates rendering offload for low-power XR devices [?]. UnifiedXRMotion’s local/remote placement abstraction is narrower: it makes node placement explicit and exposes application-level latency traces for motion-buffer transmission.

3 Schema-Based Motion-Data Contract

UnifiedXRMotion represents skeletal motion through a *schema* rather than through a hard-coded claim about the human body or avatar rig. A schema defines addressable motion slots, parent relations, and optional region views. A slot is an indexable location in the motion representation, such as a head, wrist, fingertip, tracker, prop, facial control, or application-specific joint. Each slot may carry pose components such as position, rotation, velocity, and angular velocity. Each component is guarded by a binary validity flag, so a tracker can publish only the data it observes and downstream nodes can avoid reading absent components.

3.1 Contract Definition and Invariants

A motion-data contract can be described as a tuple

$$\mathcal{C} = (S, P, R, K, V), \quad (1)$$

where $S = \{s_0, \dots, s_{n-1}\}$ is a finite set of motion slots, $P \subseteq S \times S$ is an optional parent relation, $R = \{r_1, \dots, r_m\}$ is a set of named region views with each $r_j \subseteq S$, K is the set of pose-component kinds, and V stores component-level validity. A slot is a stable address in the contract; it is not itself a claim that the underlying tracker directly observes that body part. A region view is a named projection over global slot identities, such as a hand view, upper-body view, lower-body view, or full-body view.

For each slot s_i , the runtime stores a pose record

$$m_i = (p_i, q_i, v_i, \omega_i), \quad (2)$$

Table 1. Contract invariants used by adapters, consumers, and network wrappers. These are design constraints of the runtime contract, not claims of universal device support.

Invariant	Requirement	Why it matters
Subset publication	A source may publish any subset of schema slots.	Headsets, hands, controllers, trackers, and vision sources can expose different body coverage without requiring fake complete skeletons.
Component validity	A consumer may read a component only when its validity flag is true.	Missing rotations, positions, velocities, or angular velocities are explicit data states rather than implicit errors.
Stable slot identity	Region views preserve global slot identities.	A hand-only node and a full-body node can refer to the same wrist slot without translating local indices.
Adapter normalization	SDK-specific coordinate conversion and joint-order mapping are completed before publication.	Downstream consumers depend on contract semantics, not vendor coordinate conventions.
Schema agreement	Adapters and consumers must agree on the schema version or layout used by the application.	The contract is a runtime agreement, not an automatic discovery mechanism for arbitrary avatar rigs.
Network preservation	Serialization and network wrappers preserve slot identity and validity state.	Local and remote placements can exchange partial motion records without changing the meaning of missing data.

where p_i is position, q_i is orientation, v_i is linear velocity, and ω_i is angular velocity in the prototype record. The corresponding validity mask is

$$V_i = (b_i^p, b_i^q, b_i^v, b_i^\omega), \quad b_i^k \in \{0, 1\}. \quad (3)$$

A component m_i^k is consumable only when $b_i^k = 1$. This rule is the operational core of the contract: partial trackers do not need to synthesize missing components merely to satisfy a fixed skeleton layout, and consumers must not silently assume unavailable components.

Table ?? separates the general contract from the current prototype instantiation. This separation is essential. The prototype’s 77-slot layout is useful evidence that the contract can express a full-body avatar pipeline, but it is not a universal skeleton standard.

Partial tracking is the normal case in XR. A headset and controllers provide only a few rigid poses; a hand tracker may provide fingers but no lower body; a vision model may provide upper-body positions with unreliable rotations; and a full-body SDK may infer unobserved joints internally. UnifiedXRMotion therefore treats absence as first-class data rather than as an error. A downstream node that needs rotations can reject or ignore slots whose rotation flag is absent; a filter that only needs positions can still operate when rotations are missing.

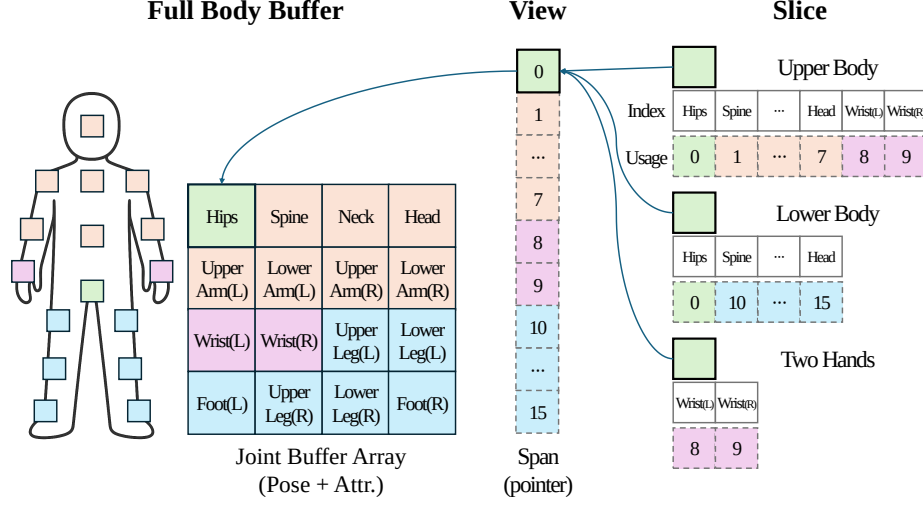


Fig. 1. Prototype buffer and region-view instantiation of the schema-based motion-data contract. The figure illustrates how a concrete full-body layout is exposed through buffer views for downstream nodes. The slot count shown in the prototype is an implementation example, not a universal skeleton standard or the conceptual contribution.

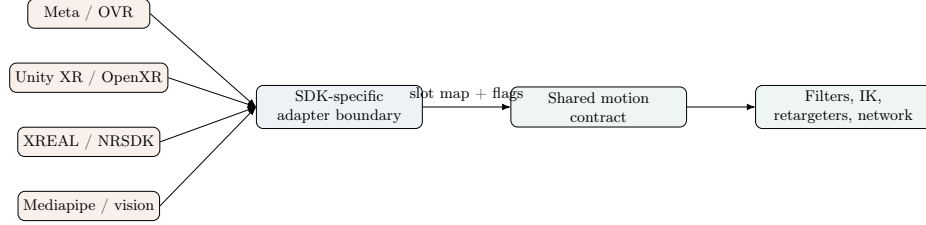


Fig. 2. Adapter boundary. Native tracking sources remain SDK-specific, but each adapter maps observed data into the same schema-based motion contract.

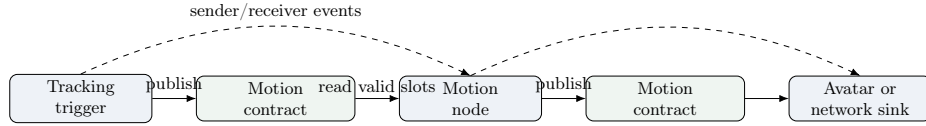
4 Adapter Boundary

UnifiedXRMotion uses SDK-specific adapters to map native tracking outputs into the motion-data contract. The central boundary is intentionally small: an adapter initializes the native source, polls or receives tracking data, exposes the slots it can publish, and returns pose records with validity flags. The adapter code is not the research novelty; much of it is necessarily vendor-specific wrapper code. The contribution is that all adapters target the same schema and validity semantics.

Table ?? preempts a likely reviewer objection. UnifiedXRMotion is partly an integration layer, because any practical XR system must interface with SDKs. The paper’s claim is not that the wrappers are novel. The claim is that a small adapter boundary plus a validity-aware contract prevents the rest of the motion

Table 2. Motion contract concepts. The prototype instantiation is evidence of implementation scope, not the conceptual boundary of the contract.

Concept	General definition	Prototype instantiation	Limitation
Skeleton schema	Defines addressable motion slots and optional parent relations.	Full-body schema with 77 slots covering torso, limbs, head, and articulated hands.	Schema agreement is required between adapters and consumers.
Region view	Named subset of slots for partial-body processing.	Views for hands, upper body, lower body, and full body.	Views are implementation-defined rather than discovered from arbitrary rigs.
Pose record	Optional components attached to a slot.	Position, rotation, velocity, and scalar angular velocity fields in a per-slot record.	Angular velocity is under-specified for full 3D rotation if represented as a scalar.
Validity	Binary component-level availability.	Flags indicate whether position, rotation, velocity, or angular velocity is present.	Continuous confidence is not preserved in the current contract.
Buffer view	In-process access to preallocated records.	Kinematic buffer views expose whole-body or region-specific views.	Zero-copy applies only to in-process views, not network transport.
Wire format	Serialization follows the same validity contract.	Invalid components can be omitted during serialization.	This paper treats serialization as semantic preservation rather than as a compression benchmark.

**Fig. 3.** Actual runtime model. UnifiedXRMotion uses trigger-based scheduling and sender/receiver event propagation over a shared motion contract. It should not be interpreted as a general DAG execution engine.

pipeline from depending directly on each SDK’s joint order, coordinate conventions, validity semantics, and update conventions. Tables ?? and ?? make this boundary concrete: source-specific conversions occur before publication, while downstream nodes consume schema views and validity masks.

5 Runtime and Local/Remote Node Placement

The runtime is an event-driven motion-node pipeline. A trigger list identifies root nodes that poll or receive new motion data. When a root node updates, it invokes a sender event; downstream receivers subscribe to that event, read the upstream motion contract through buffer views, write their own output records, and publish another update. This design is intentionally simple and Unity-friendly. It should not be described as topological graph scheduling. There is no general cycle detector or graph optimizer in the current implementation.

Table 3. Prototype adapter matrix. The table reports implementation coverage, not equal experimental validation.

Adapter/source	Native SDK or source	Motion coverage	Mapping into contract	Validation status and limitation
Meta body/hand	OVR / OpenXR paths	Body, hands, HMD, controllers	Native bone IDs and device poses are mapped to schema slots.	Implemented and used in prototype traces; remains vendor-specific wrapper code.
XREAL hand	NRSDK / XR Hands path	Hands and head-relative context	Hand joints are mapped to hand-region slots with vendor-specific offsets.	Implemented; previous video quality issues mean it should not support avatar-quality claims.
Unity XR	Unity XR / OpenXR input	HMD, controller, hand sources	Generic Unity poses are published into selected slots.	Implemented as adapter evidence; not a proof of full OpenXR coverage.
Vive / OpenXR	Vive OpenXR hand tracking	Hand joints	Vendor hand joints are converted to hand-region slots.	Implemented; not evaluated as a separate user study condition.
Mediapipe	Vision-based landmarks	Hands or upper body	Landmarks are converted into slot positions and binary validity.	Implemented; continuous confidence is quantized away.
Android AR / transform / replay / dummy	Mobile AR camera, scene transforms, recorded data, synthetic sources	Application-specific partial motion	Poses are published into selected slots for testing or replay.	Useful for testing and demos; not hardware-generalization evidence.

For client-server deployments, UnifiedXRMotion provides a network wrapper around selected motion nodes. The wrapper exposes placement states describing where the producer, node execution, and consumer reside. The structural claim is that a node can be wrapped for different placements without rewriting the node implementation. This is not a claim that all placements are deterministically equivalent or faster; equivalence and performance require separate measurement.

The current latency utility logs application-level motion-pipeline or RPC timing. It does not include sensor exposure, SDK internal sampling, Unity render submission, compositor scheduling, display scan-out, or photon emission. Table ?? defines the safe interpretation.

6 Implementation

The prototype is implemented as a Unity package. Its core implementation separates four concerns. First, motion slots are stored in preallocated buffers. Region views expose subsets of the schema, allowing a hand-only node, upper-body node, or full-body node to operate on the subset it needs. In-process view access avoids copying the entire pose record for each node. Network transmission necessarily serializes records and is not zero-copy. Second, SDK-specific adapters

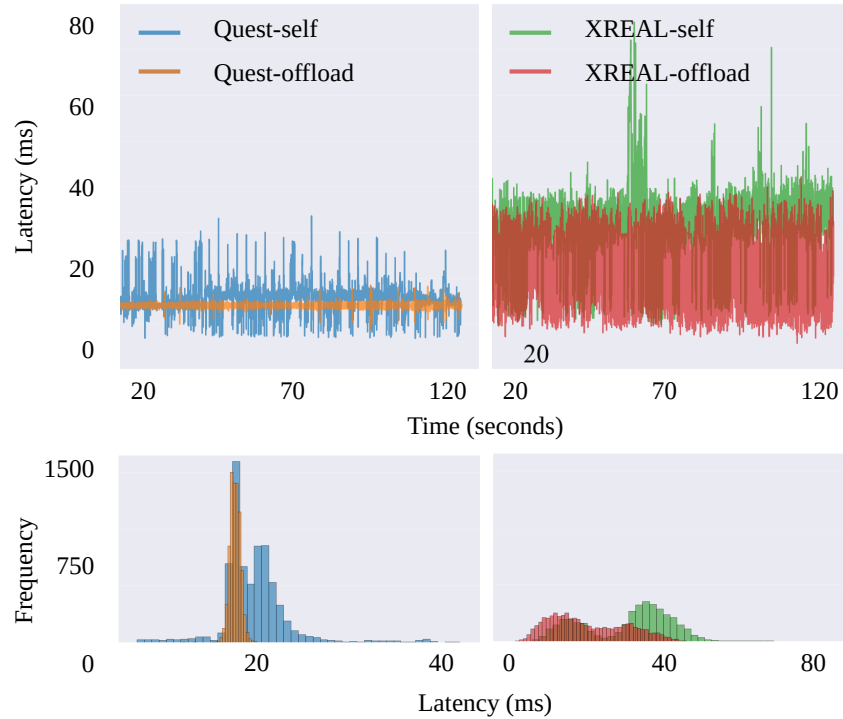


Fig. 4. Local/remote node-placement abstraction in the prototype. The same class of motion-node pipeline can be configured for local or remote execution contexts through a wrapper. The figure illustrates a placement mechanism, not deterministic equivalence or a general latency-improvement claim.

publish native tracking data into the shared contract. Some adapters are direct wrappers over vendor SDKs; others support synthetic transforms, replay, or vision-based sources. Third, the prototype includes IK, retargeting, codebook, filtering, record, and replay nodes. These nodes demonstrate that the contract can serve multiple consumers, but they are not claimed as new algorithms. Textbook solvers such as FABRIK [?] and codebook-style control inspired by prior work [?] should be cited as algorithmic background rather than framed as the paper’s novelty. Fourth, the implementation includes CSV and histogram export utilities for timing traces, plus record/replay utilities that can support reproducible evaluation.

6.1 Representative Consumer: Retargeting Through Contract Views

Retargeting is used here as a representative consumer of the contract rather than as a claimed algorithmic contribution. A retargeter subscribes to a schema

view, checks component validity, and maps the valid source transforms into an avatar-specific rig. For body skeleton retargeting, let $\mathbf{T}_{j,\text{src}}^0$ and $\mathbf{T}_{j,\text{src}}(t)$ denote the homogeneous transforms of source joint j at the canonical pose and at time t , and let $\mathbf{T}_{j,\text{tgt}}^0$ denote the target avatar’s canonical transform. A representative relative-transform mapping is

$$\mathbf{T}_{j,\text{tgt}}(t) = \mathbf{T}_{j,\text{src}}(t) \mathbf{A}_j \mathbf{T}_{j,\text{tgt}}^0, \quad (4)$$

where $\mathbf{A}_j = (\mathbf{T}_{j,\text{src}}^0)^{-1}$ is a conceptual alignment from the source canonical frame. In the implementation, equivalent alignment is applied through transform composition rather than through a stored separate variable.

Hand retargeting illustrates why source-specific frames should be normalized before downstream consumption. XR hand SDKs expose inconsistent local frames, so the adapter/consumer boundary constructs anatomical reference frames from observable hand geometry. Let $\mathbf{R}_i = [\hat{\mathbf{x}}_i, \hat{\mathbf{y}}_i, \hat{\mathbf{z}}_i]$ denote a reference frame for hand segment i , with origin, forward, and lateral reference points $\mathbf{p}_i^{(o)}$, $\mathbf{p}_i^{(f)}$, and $\mathbf{p}_i^{(l)}$. A representative construction is

$$\hat{\mathbf{x}}_i = \frac{\mathbf{p}_i^{(l)} - \mathbf{p}_i^{(f)}}{\|\mathbf{p}_i^{(l)} - \mathbf{p}_i^{(f)}\|}, \quad (5)$$

$$\hat{\mathbf{z}}_i = \frac{\mathbf{p}_i^{(f)} - \mathbf{p}_i^{(o)}}{\|\mathbf{p}_i^{(f)} - \mathbf{p}_i^{(o)}\|}, \quad (6)$$

$$\hat{\mathbf{y}}_i = s \frac{\hat{\mathbf{z}}_i \times \hat{\mathbf{x}}_i}{\|\hat{\mathbf{z}}_i \times \hat{\mathbf{x}}_i\|}, \quad (7)$$

with $s = +1$ for left hands and $s = -1$ for right hands. This example is included to show how contract consumers can reuse the same hand-region view after adapter normalization; it should not be read as a novel retargeting method.

7 Evaluation and Case Study Evidence

The evidence is scoped to the systems contribution claimed in this paper. We report implementation evidence for the contract and adapter boundary, a within-subjects setup-burden case study, and application-level traces illustrating local/remote node placement. These results support the use of a shared runtime contract for selected heterogeneous avatar-motion pipelines; they are not presented as a complete performance evaluation, a vendor-native runtime baseline, an avatar-quality study, or evidence of universal device coverage.

7.1 Contract and Adapter Coverage

Tables ??, ??, ??, ??, ??, and ?? summarize the implementation evidence for the motion contract, adapter boundary, and original-vs-wrapper separation. The important result is not the number of slots or adapters; it is the separation of

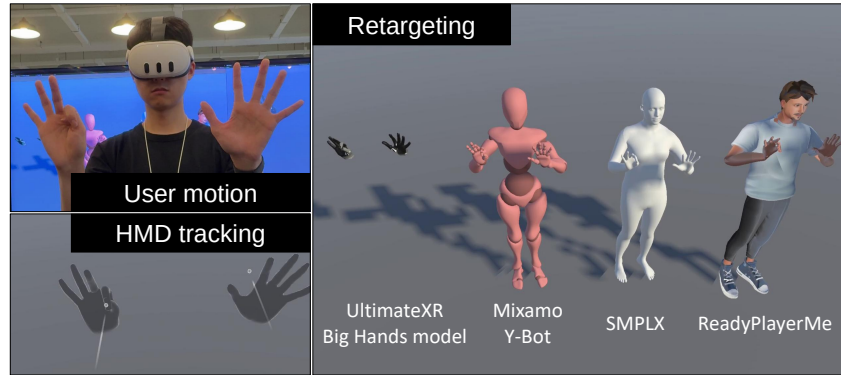


Fig. 5. Representative retargeting consumers reading from contract views and writing to heterogeneous avatar rigs. The figure illustrates consumption of the shared motion contract, not a new retargeting algorithm.

concerns. Device-specific logic is localized in adapters, while downstream nodes operate on the shared contract. This supports the claim that UnifiedXRMotion implements a concrete schema-based contract and demonstrates it with selected sources. It does not prove generality across all XR devices.

7.2 Authoring Case Study: Setup Burden

We conducted a within-subjects Unity setup task comparing a UnifiedXRMotion workflow against a vendor SDK workflow. Participants configured the same scene-level goal in both conditions: making custom OpenXR hand prefabs follow simulated hand motion and making the same Y Bot avatar follow simulated full-body motion. Both conditions used the Meta XR Simulator for verification, including random full-body motion and hand motion or pinch gestures. The study used Unity 6000.0.33f1 and Meta Core, Movement SDK, and XR Simulator version 78.0.0.

To avoid conflating pipeline assembly with mesh-specific hand-axis calibration, custom hand retargeting and axis alignment were completed before each session. This made the comparison conservative with respect to UnifiedXRMotion’s automatic hand-retargeting support, but it also means the study does not measure the full cost of integrating a previously unseen hand mesh. Participants were instructed to prioritize correctness and completion rather than speed. We therefore interpret completion time as a coarse measure of procedural setup burden, not as a pure speed-optimization measure.

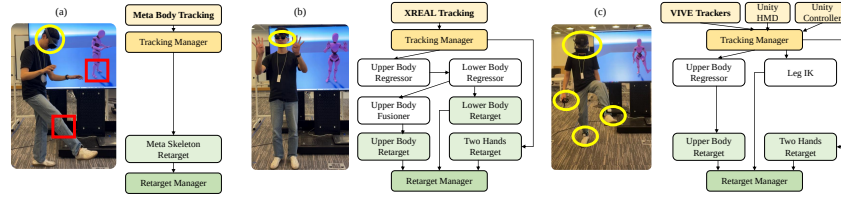


Fig. 6. Representative setup workflow used in the authoring case study. The figure is used to clarify the procedural configuration burden measured by the study; it is not a general proof of developer productivity or long-term maintainability.

Table 8. Task-equivalence summary for the authoring case study. SDK A corresponds to UnifiedXRMotion; SDK B corresponds to the vendor SDK workflow.

Aspect	UnifiedXRMotion condition	Vendor SDK condition	Interpretation / limitation
Scene assets	Task scene with Y Bot, Hands, and OpenXR custom hand prefabs.	Matched task scene with the same semantic assets.	Same scene-level goal, not identical procedures.
Full-body goal	Add Motion Avatar, MotionSystem, Meta body tracking, and full-body retarget node.	Configure camera rig, body tracking support, and Movement SDK retargeter.	Measures setup burden for one prepared full-body task.
Hand goal	Add Motion Avatar, MotionSystem, Meta hand tracking, and two-hands retarget node.	Configure interaction prefabs, synthetic hands, custom skeleton visuals, and data modifiers.	Vendor task has more visible prefab wiring; that is part of the compared workflow.
Verification	Meta XR Simulator random body motion and hand/pinch checks.	Same simulator-based behavioral check.	Verifies scene behavior, not avatar-motion quality.
Hand-axis alignment	Pre-completed before the session.	Pre-completed before the session.	Avoids a confound but excludes full new-mesh integration cost.
Completion criterion	Scene behavior matches reference clips.	Scene behavior matches reference clips.	Completion depends on procedural correctness.

Table 9. Within-subjects setup-burden results ($N = 19$). Values are mean $\pm$ standard deviation. NASA-TLX is the 0–100 workload score after reversing the performance subscale so that higher values consistently indicate higher workload. Completion time is interpreted as procedural setup burden because participants prioritized correctness and completion rather than speed.

Statistic	Completion time (s)	NASA-TLX	SUS
UnifiedXRMotion ($M \pm SD$)	564.03 $\pm$ 176.58	30.53 $\pm$ 17.55	73.42 $\pm$ 17.08
Vendor SDK ($M \pm SD$)	972.89 $\pm$ 243.36	49.74 $\pm$ 18.29	34.47 $\pm$ 13.91
95% CI (UXM – SDK)	[−493.25, −324.48]	[−24.87, −13.55]	[28.69, 49.21]
$t(df)$	−10.18 (18)	−7.13 (18)	7.98 (18)
p	< .0001	< .0001	< .0001
Hedges’ g_{av}	−1.84	−1.03	+2.39

CIs, t , p , and effect sizes are computed on paired differences. These statistics are carried over from the existing study analysis; no new user-study measurements were added.

Table ?? suggests that the UnifiedXRMotion workflow reduced procedural setup burden and improved subjective usability for the tested task. The result should not be generalized to all XR development tasks: participants did not author new adapters, did not integrate a previously unseen hand mesh, and did not evaluate long-term maintainability. The completion-time result also showed a sequence-sensitive pattern across the counterbalanced orders (mean difference −307.6 s for AB and −521.4 s for BA; Welch test $t(16.94) = 3.32$, $p = .0040$), so the time difference should be treated as evidence of procedural burden in this task rather than as a stable developer-speed estimate. Sequence sensitivity was not observed for NASA-TLX ($t(16.10) = -0.31$, $p = .763$) or SUS ($t(16.99) = 1.53$, $p = .145$).

7.3 Case Study: Local and Remote Motion-Node Placement

UnifiedXRMotion allows selected motion nodes to be configured under local or remote placement. We use the latency traces in this subsection to illustrate this capability: a motion-buffer path can be placed differently and inspected with the prototype’s logging utilities. These measurements are intended to characterize behavior exposed by the node-placement abstraction, not to establish superiority over vendor-native pipelines.

The metric is application-level motion-pipeline latency. It includes runtime and network-wrapper timing captured by the prototype, but excludes tracker capture, SDK internal sampling, compositor scheduling, display scan-out, and photon emission. It is therefore not input-to-photon or motion-to-photon latency. Table ?? reports the available traces.

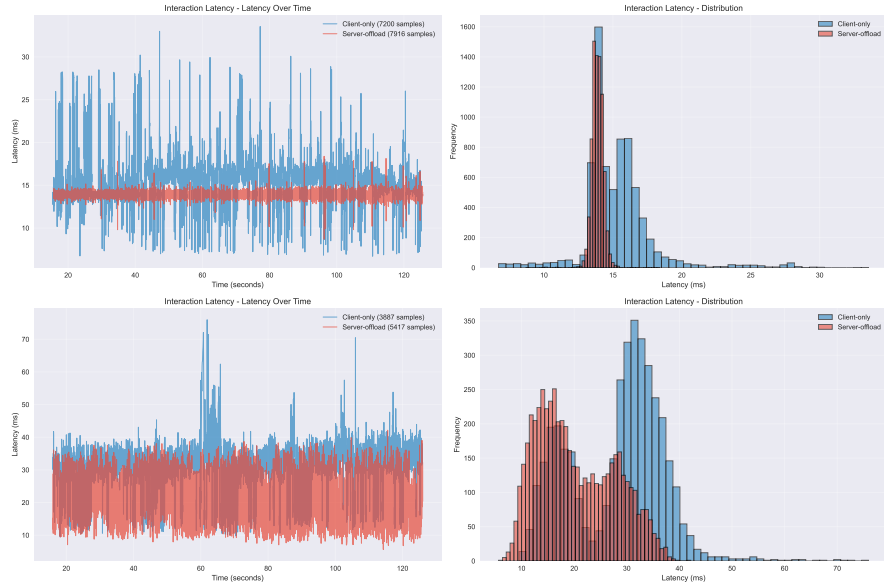


Fig. 7. Application-level motion-pipeline latency traces under local and remote node-placement configurations. The plots characterize runtime behavior exposed by the placement abstraction and exclude tracker capture, SDK-internal sampling, compositor scheduling, display scan-out, and photon emission. They should not be interpreted as input-to-photon measurements or vendor-native baselines.

Table 10. Application-level motion-pipeline latency under local and remote node-placement configurations. Each condition is a trace, not an independent repeated-run distribution.

Device	Placement	n	Mean	p50	p95	p99
Meta Quest 3	Local path	7859	15.27	14.86	18.98	26.96
Meta Quest 3	Remote path	8627	13.91	13.88	14.52	14.91
XREAL X4000	Local path	4190	28.64	30.46	40.21	50.90
XREAL X4000	Remote path	5913	20.30	18.53	33.34	36.29

The results show that placement choices can change application-level latency distributions in the tested prototype, motivating explicit placement as a runtime concern. The traces do not explain the cause of each distributional difference. The current instrumentation does not isolate serialization time, network transport, SDK buffering, scheduler effects, device temperature, or vendor-native runtime behavior. A stronger performance paper would require repeated runs, vendor-native baselines, component-level timing, synchronized clocks, and motion-to-photon instrumentation.

8 Discussion

Why a motion contract is the research object. In avatar-motion pipelines, the hard engineering problem is not only acquiring tracking data; it is keeping track of what a source actually observed, representing that partial data consistently, and making downstream nodes robust to missing components. UnifiedXRMotion addresses that problem through schema-defined slots, component-level validity, adapter-localized coordinate conversion, and reusable region views.

What the abstraction enables. The contract lets adapters, filters, retargeters, and network serializers share a common representation. It also makes missing data explicit. This is valuable for XR because different trackers expose different body regions and component reliability. A hand source, a sparse controller source, a replay stream, and a vision source can all publish partial motion into the same runtime representation, provided that they agree on the schema.

Where the abstraction is still weak. The contract is not yet a stable standard. The current prototype does not implement full timestamp propagation, explicit coordinate-frame metadata in every record, continuous confidence, or schema-version negotiation. These omissions matter. Without timestamps, latency and synchronization claims are limited. Without coordinate-frame metadata, frame conversions remain adapter-specific and easy to misconfigure. Without continuous confidence, vision trackers lose information when confidence is collapsed into binary validity. Without schema versioning, external tools cannot safely exchange saved motion streams across versions.

Local/remote placement remains structural. The network wrapper is a useful systems abstraction, but structural reuse is not the same as equivalence. A wrapped node may receive data at different times, serialize different subsets, or run under different scheduling conditions. A future version should run the same recorded motion stream through local and remote placements and report per-slot output differences.

9 Limitations

This work has significant limitations. First, it does not measure input-to-photon or motion-to-photon latency. The reported traces exclude sensor sampling, SDK internal processing, compositor scheduling, rendering, display scan-out, and photon emission. Second, the prototype does not prove universal device support. It implements selected adapters, but each new tracker still requires a schema mapping, coordinate conversion, and validation. Third, the current 77-slot schema is one prototype instantiation; arbitrary avatar rigs require schema agreement and retargeting between the runtime representation and the avatar-specific skeleton. Fourth, UnifiedXRMotion does not introduce new tracking, IK, or retargeting algorithms. The included solvers and retargeters are representative consumers of the contract.

The authoring case study also has limits. Hand-axis alignment and mesh-specific retargeting were pre-completed before sessions, so the study isolates pipeline assembly rather than full new-avatar integration. Participants were instructed to prioritize correctness and task completion, so completion time should be interpreted as procedural setup burden, not pure speed. The study does not measure adapter-authoring effort or long-term maintainability. Finally, this paper does not include a bytes-per-frame wire-format benchmark, a serialization round-trip test, a deterministic local/remote equivalence check, or a vendor-native runtime latency baseline. These are appropriate future validations, but they are not claimed here.

10 Conclusion

UnifiedXRMotion reframes heterogeneous XR avatar-motion integration as a contract problem. Rather than proposing a universal skeleton, a new tracker, or a DAG execution engine, it defines a schema-based validity-aware representation in which trackers publish partial motion into addressable slots and downstream nodes consume only valid components. The prototype demonstrates this contract through selected SDK adapters, event-driven motion nodes, representative retargeting and IK consumers, and a local/remote node-placement wrapper. The current evidence is not sufficient for broad latency or avatar-quality claims, but it supports a narrower EuroXR contribution: a concrete runtime contract and abstraction boundary for composing heterogeneous XR avatar-motion pipelines, with supporting evidence from a setup-burden case study and local/remote placement traces.

Table 4. Examples of source-specific coordinate and data-format differences handled at the adapter boundary. The table is implementation evidence for localization of heterogeneity, not a claim of new tracking.

Source / SDK	Native format	Native coordinate convention	Contract mapping	Published region	Valid components / limitation
Unity OpenXR	26×6 DoF hand joint poses	Left-handed, Z-forward tracking space	Direct slot mapping for hand joints.	Hand slots	Position and rotation valid where provided by Unity XR Hands. Position and rotation valid after conversion; remains SDK-specific wrapper logic.
Meta OpenXR	26×6 DoF hand joint poses	Right-handed, Z-backward	Z-flip for position and rotation before publication.	Hand slots	Rotation is native; positions are reconstructed by forward kinematics. Position and rotation valid after vendor-specific offsets.
Meta OVR	24×3 DoF local joint rotations plus 6DoF root pose	Right-handed, X-forward local frames	Right-to-left conversion, X-to-Z remapping, and local-to-world reconstruction before slot publication.	Hand slots; body/HMD/controller paths in separate adapters	Position and rotation valid after vendor-specific offsets.
XREAL NRSdk	23×6 DoF joint poses	Left-handed, Y-forward	Y-to-Z axis rotation and hand-region slot mapping.	Hand slots and head-relative context	Position and rotation valid after vendor-specific offsets.
XREAL OpenXR	26×6 DoF joint poses	Left-handed, Z-forward	Direct OpenXR-style hand mapping.	Hand slots	Position and rotation valid where the run-time

Table 5. Same-consumer, different-source evidence. The goal is to show that source-specific logic is localized at the adapter boundary while downstream consumers reuse contract views.

Pipeline / source	Adapter-specific code	Shared contract view	Reused downstream consumer	Remaining source-specific concern
Meta body to avatar	OVX/OpenXR body mapping and device-pose extraction.	Full-body or upper-body schema view.	Full-body retarget node and avatar binding.	Vendor body inference quality and coordinate calibration. Vendor hand-joint offsets and tracking quality.
XREAL hand to avatar	NRSDK/OpenXR hand conversion and head-relative adjustment.	Two-hands region view.	Two-hands retarget node.	Which sparse slots are physically observed.
Unity XR HMD/controllers to estimator	Generic Unity pose handlers for HMD and controllers.	Sparse head/wrist/controller slots.	Upper-body regressor, filter, or replay node.	Rotation confidence is not preserved as continuous confidence in the current prototype.
MediaPipe position stream to processing nodes	Camera-to-tracking normalization and landmark-to-slot mapping.	Position-valid hand or upper-body view.	Position-only filters, record/replay, and network serialization.	Fidelity depends on recorded data and schema agreement.
Replay / dummy sources to testing	Recorded or synthetic pose publication.	Same schema views as live adapters.	Filters, retargeters, and network wrapper tests.	

Table 6. Latency metric definition. The safe metric is useful for runtime debugging but must not be renamed as input-to-photon latency.

Metric	Included components	Excluded components	Safe interpretation	Unsafe interpretation
Application-level motion-pipeline latency	Runtime tick/RPC delivery path between stamped motion-buffer send and receive events, plus framework scheduling effects included in that path.	Sensor sampling, native SDK buffering, clock synchronization, rendering, compositor, scan-out, photon emission.	Descriptive timing of the prototype’s motion-buffer path under specific local/remote placements.	Input-to-photon, motion-to-photon, or proof that offloading improves perceived latency.
Frame-time logging utility	Unity frame interval or tick records exported for analysis.	Network transport decomposition and tracker capture time.	Runtime diagnostic signal.	Network latency or user-perceived latency by itself.

Table 7. Original-vs-wrapper boundary. This table states what the paper can safely claim and what it must not claim.

Component	Classification	Role in this paper	Safe claim	Unsafe claim
Schema, slot records, validity flags, buffer views	Original framework logic	Motion-data contract	Defines the runtime representation consumed by adapters and nodes.	Universal skeleton standard.
Validity-aware serialization	Original logic over networking library	Wire representation	Can omit unavailable components according to flags.	Measured payload reduction without a benchmark.
Tracking adapters	SDK-specific wrappers plus mapping logic	Evidence of adapter boundary	Map selected SDK outputs into the shared contract.	New tracking algorithms or universal device support.
Event-driven node interfaces	Original runtime logic	Composition model	Supports sender/receiver motion-node pipelines.	Full DAG scheduler or graph optimizer.
IK, retargeting, codebook consumers	Textbook, third-party, or application-specific nodes	Example consumers	Demonstrate contract consumption by motion-processing nodes.	Novel IK or retargeting contribution.
Network wrapper	Original logic over networking library	Local/remote placement	Provides four structural placement states for wrapped nodes.	Deterministic equivalence or latency improvement without measurement.
Timing utilities	Original support code	Descriptive characterization	Export application-level timing traces.	Input-to-photon or motion-to-photon measurement.